



Universidad Tecnológica Nacional
Facultad Regional Córdoba

Trabajo práctico N°2: programación de la placa BluePill en BareMetal

Ejercicio Práctico N°1: interrupciones externas y el NVIC

Técnicas Digitales II

4R2

Gastón Grasso 401892

Franco Palombo 401910

Angelo Prieto 401012

Última actualización: 8 de junio de 2026

Índice

1	Introducción	2
2	Desarrollo	3
2.1	Ejercicio Práctico N1: interrupciones externas y el NVIC	3
2.1.1	Archivo de Inicialización y Tabla de Vectores	3
2.1.2	Script de Enlazado	5
2.1.3	Código Fuente Principal en C	6
2.1.4	Relación y Flujo de Ejecución entre los Componentes	10
3	Conclusiones	11
4	Fuentes	12

1 Introducción

Este informe técnico documenta el desarrollo y la resolución del Ejercicio Práctico N° 1 del Trabajo Práctico N° 2 de la materia Técnicas Digitales II. Las actividades propuestas se llevan a cabo sobre la placa de desarrollo Blue Pill, la cual incorpora un microcontrolador STM32F103C8T6 basado en el núcleo ARM Cortex-M3. El enfoque de programación implementado es Bare Metal puro, interactuando de forma directa con los registros de los periféricos a través de punteros de memoria.

La guía de trabajos prácticos plantea una serie de cinco ejercicios orientados a comprender el funcionamiento y configuración de diversos módulos del microcontrolador:

1. Ejercicio Práctico N° 1 (Interrupciones Externas y el NVIC): Configuración del controlador de interrupciones vectorizadas y anidadas (NVIC) para gestionar eventos asíncronos provenientes de un pin de entrada (botón) mediante interrupción externa (EXTI), modificando el estado de un LED.
2. Ejercicio Práctico N° 2 (Conversión Analógica-Digital): Configuración del ADC para la lectura del sensor de temperatura interno y de un canal analógico externo.
3. Ejercicio Práctico N° 3 (Migración a CMSIS y Gestión de Tiempos): Incorporación del estándar CMSIS para mejorar la legibilidad y uso del temporizador del sistema (SysTick) para generar retardos precisos.
4. Ejercicio Práctico N° 4 (Comunicación Serie UART): Establecimiento de comunicación bidireccional con una PC mediante el periférico USART para la transmisión de estados y recepción de comandos.
5. Ejercicio Práctico N° 5 (Control de Potencia con Timers): Generación de señales de modulación por ancho de pulso (PWM) empleando los temporizadores de propósito general para variar la intensidad lumínica.

Cabe destacar que el presente documento aborda de manera exclusiva la resolución del Ejercicio Práctico N° 1. Los ejercicios subsiguientes (del 2 al 5) se desarrollarán de forma progresiva en futuros informes. El objetivo principal de esta primera etapa consiste en comprender la tabla de vectores de interrupción, el funcionamiento de la línea EXTI y la habilitación de interrupciones en el NVIC sin utilizar bibliotecas de alto nivel.

2 Desarrollo

En este capítulo se detalla la resolución técnica del primer ejercicio práctico, correspondiente al diseño e implementación del sistema de gestión de interrupciones externas utilizando la placa BluePill en un entorno Bare Metal.

2.1 Ejercicio Práctico N1: interrupciones externas y el NVIC

En este ejercicio se ha configurado el microcontrolador STM32F103C8T6 para que el LED interno (conectado al pin PC13) permanezca parpadeando periódicamente en el bucle principal, mientras que se implementan dos pulsadores externos asociados a interrupciones externas con diferentes prioridades en el NVIC:

- El Botón A (conectado en la GPIO C15) activa la interrupción de alta prioridad (EXTI15_10) que comanda el led externo 1 (conectado en la GPIO C14), haciéndolo parpadear 15 veces de forma rápida.
- El Botón B (conectado en la GPIO B0) activa la interrupción de baja prioridad (EXTI0) que comanda el led externo 2 (conectado en la GPIO B1), haciéndolo parpadear 15 veces de forma rápida.

El propósito de contar con este par de botones y leds es demostrar físicamente el mecanismo de la anidación de interrupciones en el NVIC, observando cómo la interrupción de alta prioridad puede interrumpir la de baja prioridad, deteniendo temporalmente su ciclo de parpadeo.

Para llevar a cabo la resolución de este ejercicio práctico, el proyecto se divide en tres archivos fundamentales: el código de arranque en ensamblador (`crt.s`), el script de enlazado (`linker.ld`) y el código fuente principal en C (`main.c`). A continuación, se detalla el análisis de cada uno de ellos y la relación funcional que existe entre sí.

2.1.1 Archivo de Inicialización y Tabla de Vectores

El archivo `crt.s`, cuyo código se expone en el listado 2.1, representa el código de inicio (startup) del sistema. Al energizar el microcontrolador, la CPU del Cortex-M3 inicializa el puntero de pila (Stack Pointer) con el valor ubicado en la dirección de memoria `0x08000000` (inicio de la memoria FLASH) y salta a la dirección del vector de reinicio (Reset Vector) ubicada en `0x08000004`.

En este archivo se define la sección `.isr_vector`, la cual contiene la tabla de vectores de interrupción requerida por la arquitectura ARM Cortex-M3 [Cor]. Las primeras posiciones corresponden a excepciones internas del núcleo (Reset, NMI y Hard Fault). Las posiciones del núcleo restantes (de la 4 a la 15) se rellenan con ceros mediante la directiva `.rept 12`, ya que no son utilizadas en esta aplicación.

A continuación de las excepciones del núcleo, se posicionan los vectores de interrupción específicos de los periféricos de la placa STM32. Para demostrar el manejo de prioridades e interrupción anidada, se implementan dos controladores de interrupción diferentes: EXTIO_IRQHandler (asociado a la línea EXTIO, posición física 6 de los periféricos y 22 global) y EXTI15_10_IRQHandler (asociado a las líneas EXTI15 a EXTI10, posición física 40 de los periféricos y 56 global). Para ubicarlos de forma exacta en la FLASH, se rellenan las posiciones intermedias con ceros usando las directivas `.rept 6` y `.rept 33`.

El punto de entrada físico del firmware es el manejador de reset `_reset`, el cual llama a la función principal en C mediante la instrucción de ramificación con enlace `bl main`. Si la función principal retornara, el microcontrolador quedaría en un bucle infinito para evitar comportamientos erráticos.

```

1  .syntax unified
2  .cpu cortex-m3
3  .thumb
4  .extern _esstack
5
6  .section .text.manejadores
7
8  // Función por defecto si ocurre una interrupción no implementada
9  .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b .
13
14 // Definición de vectores como débiles (weak)
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak HardFault_Handler
18 .thumb_set HardFault_Handler, Default_Handler
19 .weak EXTIO_IRQHandler
20 .thumb_set EXTIO_IRQHandler, Default_Handler
21 .weak EXTI15_10_IRQHandler
22 .thumb_set EXTI15_10_IRQHandler, Default_Handler
23
24 // Tabla de vectores de interrupción
25 .section .isr_vector,"a",%progbits
26 .word _esstack           // Posición 0: Puntero de pila
27 .word _reset            // Posición 1: Vector de Reset
28 .word NMI_Handler       // Posición 2: NMI
29 .word HardFault_Handler // Posición 3: Falla de hardware
30
31 // Rellena las posiciones de excepciones del núcleo restantes (de la 4 a la 15)
32 .rept 12
33 .word 0
34 .endr
35
36 // Rellena los primeros periféricos (IRQ 0 a 5, posiciones 16 a 21)
37 .rept 6
38 .word 0
39 .endr
40 .word EXTIO_IRQHandler    // Posición 22: Vector EXTIO (IRQ 6)
41
42 // Rellena los periféricos restantes hasta antes de EXTI15_10 (IRQ 7 a 39, posiciones 23 a 55)
43 .rept 33
44 .word 0
45 .endr
46 .word EXTI15_10_IRQHandler // Posición 56, dirección física exacta para la línea EXTI15_10
47
48 // Sección de código de Reset
49 .section .text.reset
50 .thumb_func

```

```
51 .global _reset
52 _reset:
53     bl main
54     b .
```

Listing 2.1: Archivo de inicialización y tabla de vectores (`crt.s`).

2.1.2 Script de Enlazado

El archivo `linker.ld`, presentado en el listado 2.2, es el script de configuración del enlazador (GNU Linker). Su función principal es indicar cómo deben distribuirse las secciones de código y datos dentro del espacio de direcciones de memoria del microcontrolador.

En primer lugar, define el bloque `MEMORY`, especificando que el chip STM32F103C8T6 posee una memoria FLASH de lectura y ejecución (`rx`) de 64 kB con origen en `0x08000000`, y una memoria RAM de lectura, escritura y ejecución (`xrw`) de 20 kB con origen en `0x20000000`.

El símbolo `_esstack` se calcula sumando el origen de la RAM con su longitud, apuntando así a la última posición de la memoria RAM (pila invertida o descendente). Este símbolo es exportado y consumido directamente en la primera posición de la tabla de vectores en `crt.s`.

Dentro del bloque `SECTIONS`, la directiva `KEEP(*(.isr_vector))` asegura que la sección de la tabla de vectores de interrupción no sea eliminada por optimizaciones del compilador y que se ubique de manera obligatoria al principio de la memoria FLASH. Posteriormente, se disponen las secciones de código y constantes (`.text` y `.rodata`) en la FLASH, mientras que las secciones de variables globales (`.data` y `.bss`) se mapean para su uso en la RAM.

```
1 MEMORY
2 {
3     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
4     RAM (xrw)  : ORIGIN = 0x20000000, LENGTH = 20K
5 }
6
7 /* Punto de entrada al arrancar */
8 ENTRY(_reset)
9
10 /* Dirección final de la RAM para inicializar la pila */
11 _esstack = ORIGIN(RAM) + LENGTH(RAM);
12
13 SECTIONS
14 {
15     /* Asegura la colocación de la tabla de vectores al inicio de la FLASH */
16     .isr_vector :
17     {
18         KEEP(*(.isr_vector))
19     } > FLASH
20
21     /* Código del programa y constantes */
22     .text :
23     {
24         *(.text)
25         *(.text*)
26         *(.rodata)
27         *(.rodata*)
28     } > FLASH
29
30     /* Direcciones de control para variables globales */
31     _sidata = LOADADDR(.data);
32
33     .data :
```

```
34 {
35     _sdata = .;
36     *(.data)
37     *(.data*)
38     _edata = .;
39 } > RAM AT> FLASH
40
41 .bss :
42 {
43     _sbss = .;
44     *(.bss)
45     *(.bss*)
46     *(COMMON)
47     _ebss = .;
48 } > RAM
49 }
```

Listing 2.2: Script de enlazado de memoria (`linker.ld`).

2.1.3 Código Fuente Principal en C

El archivo `main.c`, expuesto en el listado 2.3, implementa la inicialización de los registros de control y la rutina de servicio para procesar las interrupciones externas. Al no contar con los archivos de cabecera estándar del fabricante, se definen de manera explícita las direcciones físicas de los periféricos basándose en el manual de referencia [Stm].

Mapeo de Registros

- **GPIOB:** Con dirección base en `0x40010C00`, define los registros `GPIOB_CRL` y `GPIOB_ODR` para configurar el segundo botón (PB0) como entrada con pull-up y el segundo led externo (PB1) como salida digital push-pull.
- **GPIOC:** Dirección base en `0x40011000`, define el registro de configuración alto `GPIOC_CRH` para los pines 13, 14 y 15, `GPIOC_ODR` para el manejo de los estados de los pines de salida, y `GPIOC_BSRR` para la alternancia atómica del led indicador interno.
- **RCC:** Dirección base `0x40021000`. Se utiliza `RCC_APB2ENR` para habilitar los relojes de los periféricos `GPIOB`, `GPIOC` y `AFIO`.
- **AFIO:** Dirección base `0x40010000`. Se incorporan los registros `AFIO_EXTICR1` y `AFIO_EXTICR4` para asociar los pines PB0 y PC15 a los canales `EXTI0` y `EXTI15` respectivamente.
- **EXTI:** Dirección base `0x40010400`. Se emplean los registros de máscara de interrupción (`EXTI_IMR`), detección por flanco de bajada (`EXTI_FTSR`) y el registro de petición pendiente (`EXTI_PR`) para limpiar el flag de interrupción.
- **NVIC:** Se emplean los registros `NVIC_ISER0` y `NVIC_ISER1` para habilitar los canales `EXTI0` (IRQ 6) y `EXTI15_10` (IRQ 40), y los registros de prioridad `NVIC_IPR1` y `NVIC_IPR10` para configurar sus prioridades relativas.

Configuración Paso a Paso en la Función Principal

Dentro del ciclo de inicialización en la función `main()`, se ejecutan las siguientes acciones en secuencia:

1. **Habilitación del reloj de periféricos:** Se configuran los bits 4, 3 y 0 de `RCC_APB2ENR` para suministrar señal de reloj a los puertos `GPIOC`, `GPIOB` y al módulo `AFIO`.
2. **Configuración de PC13, PC14 y PB1 como salidas:** Se establecen en modo push-pull de 50 MHz. PC13 y PC14 se configuran en el puerto C a través de `GPIOC_CRH`, mientras que PB1 se configura en el puerto B usando `GPIOB_CRL`. PC13 controla el led interno, PC14 el led externo 1, y PB1 el led externo 2.
3. **Configuración de entradas con Pull-up:** Se configura PC15 (Botón A) y PB0 (Botón B) en modo entrada con resistencia interna escribiendo `0x8` en sus respectivos registros de configuración y estableciendo en '1' los bits correspondientes en los registros de salida de datos (`GPIOx_ODR`) para definir las pull-up.
4. **Asociación de interrupciones externas:** Se escribe `0x2` en el campo `EXTI15` de `AFIO_EXTICR4` y `0x1` en el campo `EXTI0` de `AFIO_EXTICR1` para direccionar los pines PC15 y PB0 a las líneas de interrupción externa correspondientes.
5. **Configuración del disparo y máscara:** Se configuran los flancos descendentes escribiendo un '1' en los bits 15 y 0 de `EXTI_FTSR`, y se habilitan las líneas escribiendo en `EXTI_IMR`.
6. **Configuración de prioridades de interrupción:** Para demostrar la anidación, se le asigna prioridad alta (`0x40`) a `EXTI15_10` mediante el registro `NVIC_IPR10`, y una prioridad menor (`0x80`) a `EXTI0` a través del registro `NVIC_IPR1`.
7. **Habilitación en el NVIC:** Se escribe un '1' en el bit 6 de `NVIC_ISER0` (para activar `EXTI0`) y en el bit 8 de `NVIC_ISER1` (para activar `EXTI15_10`).

Una vez completada la inicialización, la CPU ejecuta de forma ininterrumpida el bucle principal `while(1)`, que alterna el estado del LED interno en PC13 mediante retardos por software (bucles vacíos de larga duración).

Rutina de Servicio de Interrupción (ISR) y Demostración de Prioridades

La demostración práctica del manejo de prioridades e interrupción anidada (preemption) se ejecuta al interactuar con las rutinas de servicio definidas en `main.c`:

- `EXTI15_10_IRQHandler` (Prioridad Alta - `0x40`): Al presionar el Botón A en PC15, se inicia un bucle largo observable por hardware que hace parpadear el led externo 1 (PC14) rápidamente 15 veces seguidas mediante un retraso por software.
- `EXTI0_IRQHandler` (Prioridad Baja - `0x80`): Al presionar el Botón B en PB0, se inicia un bucle largo similar que hace parpadear el led externo 2 (PB1) de manera rápida 15 veces seguidas.

El comportamiento observable del sistema valida de forma empírica la jerarquía de prioridades del NVIC:

1. Si se presiona el Botón B (baja prioridad), el led 2 en PB1 comienza a parpadear rápidamente.
2. Si durante este parpadeo se presiona el Botón A (alta prioridad), el NVIC suspende de inmediato la ejecución de la ISR de menor prioridad y salta a `EXTI15_10_IRQHandler`. Esto se evidencia físicamente porque el led 1 en PC14 comienza a parpadear de inmediato, pausando el parpadeo del led 2.
3. Al finalizar el parpadeo del led 1, el procesador retorna a la rutina de menor prioridad, reanudando el parpadeo del led 2 en el punto exacto donde fue pausado.
4. Por el contrario, si se presiona primero el Botón A, el led 1 parpadea y cualquier pulsación del Botón B no tiene efecto inmediato; la ISR de menor prioridad queda retenida en estado pendiente y solo comienza a parpadear el led 2 una vez que el led 1 ha completado sus 15 destellos.

```

1 #include <stdint.h>
2
3 typedef volatile uint32_t register_t;
4
5 // Periféricos GPIO
6 #define GPIOB_BASE_ADDRESS 0x40010C00
7 #define GPIOB_CRL      (*(register_t *) (GPIOB_BASE_ADDRESS)) // GPIOB cfg register L (0-7)
8 #define GPIOB_ODR      (*(register_t *) (GPIOB_BASE_ADDRESS + 0x0CU)) // GPIOB Output data reg
9
10 #define GPIOC_BASE_ADDRESS 0x40011000
11 #define GPIOC_CRH      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x04U)) // GPIOC cfg register H (8-15)
12 #define GPIOC_ODR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x0CU)) // GPIOC Output data reg
13 #define GPIOC_BSRR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x10U)) // GPIOC Set/Reset reg (atomic)
14
15 #define RCC_BASE_ADDRESS 0x40021000
16 #define RCC_APB2ENR      (*(register_t *) (RCC_BASE_ADDRESS + 0x18U)) // APB2 clk en reg
17
18 // Periférico AFIO (Alternate Function I/O)
19 #define AFIO_BASE_ADDRESS 0x40010000
20 #define AFIO_EXTICR1      (*(register_t *) (AFIO_BASE_ADDRESS + 0x08U)) // Mapea EXTI0 a EXTI3
21 #define AFIO_EXTICR4      (*(register_t *) (AFIO_BASE_ADDRESS + 0x14U)) // Mapea EXTI12 a EXTI15
22
23 // Periférico EXTI (External Interrupt Controller)
24 #define EXTI_BASE_ADDRESS 0x40010400
25 #define EXTI_IMR          (*(register_t *) (EXTI_BASE_ADDRESS + 0x00U)) // Interrupt Mask Register
26 #define EXTI_FTSR          (*(register_t *) (EXTI_BASE_ADDRESS + 0x0CU)) // Falling Trigger Selection
27                               Register
28 #define EXTI_PR            (*(register_t *) (EXTI_BASE_ADDRESS + 0x14U)) // Pending Register
29
30 // Periférico NVIC
31 #define NVIC_ISER0          (*(register_t *) (0xE000E100U)) // Interrupt Set-Enable Register 0 (IRQ
32                               0-31)
33 #define NVIC_ISER1          (*(register_t *) (0xE000E104U)) // Interrupt Set-Enable Register 1 (IRQ
34                               32-63)
35 #define NVIC_IPR1           (*(register_t *) (0xE000E404U)) // Interrupt Priority Register 1 (IRQ 4-7)
36 #define NVIC_IPR10          (*(register_t *) (0xE000E428U)) // Interrupt Priority Register 10 (IRQ
37                               40-43)
38
39 int main() {
40     // 1. Habilitación de clocks
41     // Enciende clock de GPIOC (bit 4), GPIOB (bit 3) y AFIO (bit 0)

```

```

38  RCC_APB2ENR |= (1 << 4) | (1 << 3) | (1 << 0);
39
40  // 2. Configuración de pines de entrada y salida
41  GPIOC_CRH &= (~(register_t) 0xF << 20)); // PC13 (led interno)
42  GPIOC_CRH |= 0x3 << 20; // Output push-pull 50MHz
43
44  GPIOC_CRH &= (~(register_t) 0xF << 24)); // PC14 (led externo 1)
45  GPIOC_CRH |= 0x3 << 24; // Output push-pull 50MHz
46
47  GPIOC_CRH &= (~(register_t) 0xF << 28)); // PC15 (pulsador 1)
48  GPIOC_CRH |= 0x8 << 28; // Input pull-up/pull-down
49  GPIOC_ODR |= (1 << 15); // Define entrada como pull-up
50
51  GPIOB_CRL &= (~(register_t) 0xFF << 0)); // PB0 (pulsador 2) y PB1 (led externo 2)
52  GPIOB_CRL |= (0x3 << 4) | (0x8 << 0); // PB1 output 50MHz, PB0 input pull-up/down
53  GPIOB_ODR |= (1 << 0); // Define entrada como pull-up
54
55  // 3. Configuración de interrupciones externas (EXTI)
56  // Mapea línea EXTI15 a Puerto C
57  AFIO_EXTICR4 |= 0x2 << 12;
58
59  // Mapea línea EXTI0 a Puerto B
60  AFIO_EXTICR1 &= ~(0xFU << 0);
61  AFIO_EXTICR1 |= (0x1U << 0);
62
63  // Configura flanco de bajada para EXTI15 y EXTI0
64  EXTI_FTSR |= (1 << 15) | (1 << 0);
65
66  // Desenmascara las líneas EXTI15 y EXTI0
67  EXTI_IMR |= (1 << 15) | (1 << 0);
68
69  // 4. Configuración de prioridades en el NVIC
70  // EXTI15_10 (IRQ 40) -> Prioridad alta: 0x40 (bits 4-7 de NVIC_IPR10)
71  NVIC_IPR10 &= ~(0xFFU << 0);
72  NVIC_IPR10 |= (0x40U << 0);
73
74  // EXTI0 (IRQ 6) -> Prioridad baja: 0x80 (bits 20-23 de NVIC_IPR1)
75  NVIC_IPR1 &= ~(0xFFU << 16);
76  NVIC_IPR1 |= (0x80U << 16);
77
78  // Habilita las interrupciones en el NVIC
79  NVIC_ISER1 |= (1 << 8); // Habilita IRQ 40 (EXTI15_10)
80  NVIC_ISER0 |= (1 << 6); // Habilita IRQ 6 (EXTI0)
81
82  while (1) {
83      // Parpadeo normal del led interno en PC13
84      GPIOC_BSRR = 1 << 13;
85      for (volatile int i = 0; i < 500000; i++);
86      GPIOC_BSRR = 1 << (13 + 16);
87      for (volatile int i = 0; i < 500000; i++);
88  }
89
90  return 0;
91 }
92
93 // ISR para EXTI15_10 (Button A en PC15, controla LED 1 en PC14)
94 void EXTI15_10_IRQHandler(void) {
95     if (EXTI_PR & (1 << 15)) {
96         // Parpadea LED 1 (PC14) rápidamente 15 veces (30 conmutaciones)
97         for (int j = 0; j < 30; j++) {
98             GPIOC_ODR ^= (1 << 14);
99             for (volatile int i = 0; i < 200000; i++); // Retardo observable
100         }
101         EXTI_PR = (1 << 15); // Limpia flag
102     }
103 }
104

```

```
105 // ISR para EXTI0 (Button B en PB0, controla LED 2 en PB1)
106 void EXTI0_IRQHandler(void) {
107     if (EXTI_PR & (1 << 0)) {
108         // Parpadea LED 2 (PB1) rápidamente 15 veces (30 conmutaciones)
109         for (int j = 0; j < 30; j++) {
110             GPIOB_ODR ^= (1 << 1);
111             for (volatile int i = 0; i < 200000; i++); // Retardo observable
112         }
113         EXTI_PR = (1 << 0); // Limpia flag
114     }
115 }
```

Listing 2.3: Código fuente principal e implementación de la ISR (main.c).

2.1.4 Relación y Flujo de Ejecución entre los Componentes

La interacción de los tres archivos de desarrollo analizados conforma la base necesaria para la correcta ejecución en Bare Metal:

1. El archivo de enlazado (`linker.ld`) asigna de manera rígida los límites de la memoria física y establece que la tabla de vectores definida en `crt.s` comience exactamente en la posición `0x08000000`.
2. El código en ensamblador (`crt.s`) posiciona la dirección física de la rutina `EXTI15_10_IRQHandler` en el vector número 40 del NVIC (posición 56 de la tabla).
3. Durante la compilación y el enlace, el compilador traduce la función definida en `main.c` en una dirección física de memoria, la cual el enlazador copia en el vector asignado dentro de la tabla de interrupciones.
4. Cuando la interrupción física se produce en el microcontrolador, el NVIC consulta la dirección de memoria asociada al vector 40 en la FLASH y salta hacia la función implementada en C, logrando la interacción asíncrona.

3 Conclusiones

Tras el análisis e implementación del Ejercicio Práctico N° 1, se concluye que se han alcanzado satisfactoriamente los objetivos propuestos. La utilización de una interrupción externa gestionada a través del NVIC y el módulo EXTI ha permitido resolver de manera eficiente la detección y el procesamiento de eventos asíncronos (la pulsación del botón en PC15), rompiendo el flujo secuencial del programa principal sin requerir un muestreo periódico (polling).

El desarrollo en un entorno Bare Metal puro (sin sistemas operativos ni capas de abstracción complejas) ha resultado fundamental para comprender de manera profunda el comportamiento a bajo nivel de la arquitectura ARM Cortex-M3. La configuración manual de registros específicos como los de habilitación de reloj en el RCC, control de puertos GPIO, asociación de líneas en el AFIO, configuración de flancos en el EXTI, y activación en el NVIC proporciona un control absoluto sobre el hardware y optimiza el uso de recursos de memoria y CPU.

Asimismo, la implementación de la tabla de vectores en el archivo de inicio en ensamblador y la definición exacta de la distribución física en el script de enlazado ponen de manifiesto la estrecha relación que existe entre el software desarrollado y la estructura del hardware subyacente. Se constata la vital importancia de limpiar el flag pendiente de interrupción en el registro correspondiente antes de finalizar la rutina de servicio, ya que la omisión de este paso invalida el comportamiento esperado del microcontrolador al generar bucles infinitos de atención.

4 Fuentes

- [Cor] *Cortex-M3 Technical Reference Manual*. r2p1. ARM Limited. 2010.
- [Stm] *RM0008 Reference manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs*. Rev 21. STMicroelectronics. 2021.